راهنمای فنی-شناختی برای داروساز vibe coder

کتاب راهنمای قیمت‌یاب دارو و مسیر ادغام با بات بله

این متن برای مطالعه عمیق نوشته شده است: نه خلاصه مدیریتی، نه dump کد. هدف این است که منطق پروژه را بفهمی، گسترش بدهی، ایده تولید کنی، و دفعه بعد بتوانی خودت به Codex توضیح بدهی پروژه چه می‌خواهد.

پروژه محلی: `C:\Users\hosse\Documents\قیمت\pricing-app`

repo مقصد: `hosseintajdari/bale-med`

آخرین commit مبنا: `6fcfa96 Add patient medicine selection flow`

سطح: فنی، داروسازی، شناختی، قابل تندخوانی سه‌ساعته

نقشه خواندن

1. این پروژه دقیقاً چیست؟
2. نقشه پروژه‌ها: pricing-app و bale-med
3. زبان دامنه
4. قیمت واحد
5. Qanoon Pharma و TTAC
6. انتخاب دارو قبل از قیمت
7. شکل دارویی و route
8. شناخت بیمار و ساده‌سازی
9. یادگیری جمعی
10. Session و callback
11. TTAC Detail Enrichment
12. Git برای این پروژه
13. منطق استفاده از ابزارها
14. تست‌ها
15. مسیر ادغام با bale-med
16. ایده‌سازی آینده
17. چک‌لیست تسلط
18. ضمیمه واژه‌نامه

پیشنهاد مطالعه: یک بار پیوسته بخوان؛ بار دوم فقط جدول‌ها، نمودارها و خلاصه انتهای فصل‌ها را مرور کن.

فصل ۱ - این پروژه دقیقاً چیست؟

قیمت‌یاب دارو فقط یک ماشین حساب نیست؛ یک مترجم بین بیمار، داروساز، منبع رسمی و نرم‌افزار داروخانه است.

اگر بیمار می‌گوید «قیمت استامینوفن چنده؟»، از نگاه فنی شاید سؤال ساده‌ای به نظر برسد. اما از نگاه داروسازی و شناختی، سؤال هنوز کامل نیست. کدام شکل؟ قرص، شیاف، قطره، شربت؟ کدام قدرت؟ ۳۲۵ یا ۵۰۰؟ کدام بسته؟ قیمت هر واحد مصرفی یا قیمت کل بسته؟ این پروژه از همین شکاف شروع می‌شود: شکاف بین عبارت کوتاه بیمار و داده‌های چندلایه دارویی.

در پروژه فعلی، ما یک هسته قیمت‌یابی بیمارمحور ساختیم که فعلاً جدا از `bale-med` زندگی می‌کند، اما در نهایت باید به همان بات بله وصل شود. بنابراین باید دو منطق را همزمان بفهمی: منطق قیمت‌یاب، و منطق بات داروخانه‌ای safety-first در `bale-med`. اولی به داروی درست و قیمت واحد فکر می‌کند؛ دومی به review، intake داروساز، پیام امن، و عدم ارائه توصیه درمانی.

نام دقیق پروژه می‌تواند چنین باشد: «موتور انتخاب و قیمت واحد دارو برای بیمار». این نام عمداً طولانی است، چون اگر فقط بگوییم قیمت‌یاب، ذهن سریع به سمت ارزان‌ترین گزینه می‌رود. اما محصول خوب برای بیمار اول باید از خطای انتخاب جلوگیری کند. قیمت بعد از هویت درست می‌آید، نه قبل از آن.

از سؤال بیمار تا پاسخ سالم

عبارت بیمار ← تشخیص ابهام ← انتخاب شکل/دوز/هویت ← قیمت واحد ← اطلاعات بیشتر از TTAC ← در صورت نیاز اتصال به review داروساز

قیمت‌یاب به‌عنوان مترجم

قاب داروسازی

بیمار زبان نسخه، جعبه یا حافظه خودش را دارد؛ داروساز زبان شکل، دوز، بسته و منبع را دارد.

قاب شناختی

سیستم باید بار تصمیم را کم کند. هر مرحله فقط یک تمایز مهم را نشان می‌دهد.

قاب فنی

module انتخاب دارو قبل از pricing قرار می‌گیرد و channel-neutral خروجی می‌دهد.

ایده مرکزی

پروژه وقتی خوب فهمیده شده که بتوانی بگویی: «ما قیمت نمی‌دهیم مگر اینکه واحد مصرف و هویت دارو برای بیمار به اندازه کافی روشن شده باشد.»

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱ - این پروژه دقیقاً چیست؟ را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱ - این پروژه دقیقاً چیست؟، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- قیمت در این پروژه خروجی نهایی است، نه ابزار انتخاب اولیه.
- هسته پروژه یک translation layer بین زبان بیمار و داده دارویی است.
- `bale-med` زمینه نهایی deployment و workflow داروخانه‌ای است.

تمرین برای تثبیت

- با مثال `ویتامین آ` توضیح بده چرا پرسش بیمار هنوز کامل نیست.
- یک مثال از دارویی بزن که قیمت بسته باعث گمراهی بیمار می‌شود.
- در یک جمله بگو چرا این پروژه نباید توصیه مصرف بدهد.

فصل ۲ - نقشه پروژه‌ها: `pricing-app` و `bale-med`

یکی موتور قیمت و انتخاب است؛ دیگری بات داروخانه‌ای بله. ادغام موفق یعنی هر کدام نقش خودش را نگه دارد.

در `pricing-app` ما روی یک مسئله باریک ولی عمیق کار کردیم: وقتی بیمار نام دارو را می‌فرستد، چطور داروی درست را پیدا کنیم و قیمت واحد مصرفی را واضح نشان بدهیم. در `bale-med` هدف بزرگ‌تر است: دریافت پیام در بله، ثبت امن پیام، جلوگیری از duplicate delivery، نگهداری در SQLite، reply امن فارسی، و مسیر review داروساز.

مطالعه `repo `hosseintajdari/bale-med` نشان می‌دهد که پروژه فعلی آن، local-first و pharmacist-centered است. فایل `app/main.py` route`های اصلی را می‌سازد: webhook، health check بله، pending reviews و mark-reviewed. فایل `app/handlers.py` پیام ورودی را parse می‌کند، review metadata می‌سازد، پیام را ذخیره می‌کند، reply امن می‌سازد و اگر لازم شد notification داروساز آماده می‌کند.

این یعنی قیمت‌یاب نباید جایگزین این safety layer شود. باید مثل یک module کمکی وارد شود: اگر پیام بیمار از جنس «قیمت دارو» یا «پیدا کردن دارو» بود، به موتور انتخاب و قیمت سپرده شود. اگر پیام از جنس دوز، تداخل، بارداری، کودک، حساسیت، عارضه یا تغییر دارو بود، همان مسیر review داروساز در `bale-med` باید جلوتر باشد.

بخش	نقش فعلی	نقش در آینده
pricing-app	انتخاب دارو و قیمت واحد	هسته قابل اتصال به بات بله
bale-med	intake و review داروساز	بدنه اصلی workflow بات
TTAC	هویت و جزئیات عمومی دارو	غنی‌سازی بعد از انتخاب دارو
Qanoon Pharma	منطق قیمت داروخانه‌ای	منبع قیمت اصلی مصرف‌کننده

ادغام آینده

- Bale webhook ← route_update در bale-med
- تشخیص intent قیمت ← medicine selection engine
- consumer unit price ← reply امن در بله

قاعده ادغام

قیمت‌یاب باید داخل مرز safety-first بات بنشیند. هر جا پیام وارد clinical judgment شد، مسیر داروساز اولویت دارد.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۲ - نقشه پروژه‌ها: `pricing-app` و `bale-med` را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۲ - نقشه پروژه‌ها: `pricing-app` و `bale-med`، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- `pricing-app` موتور تخصصی انتخاب و قیمت است.
- `bale-med` بدن اصلی بات داروخانه‌ای و محل workflow بله است.
- ادغام باید safety boundary را حفظ کند.

تمرین برای تثبیت

- سه نوع پیام بنویس: قیمت، intake عمومی، high-risk. برای هر کدام مسیر درست را بگو.
- توضیح بده چرا `pricing-app` نباید مستقیماً webhook بله را مالک شود.
- یک نام خوب برای module اتصال قیمت به `bale-med` پیشنهاد کن.

فصل ۳ - زبان دامنه: چرا واژه‌ها معماری می‌سازند؟

وقتی واژه درست انتخاب می‌کنی، کد بعدی کمتر شلخته می‌شود.

در این پروژه واژه‌نامه فقط تزئین نیست. `CONTEXT.md` مثل نقشه مفهومی پروژه است. اگر بگوییم enrichment، selection، strength، route، form، pack، unit و price هر کدام یعنی چه، بعداً کد مجبور نمی‌شود با حدس جلو برود.

تو به خاطر زمینه زبان‌شناسی شناختی، احتمالاً این بخش را سریع می‌گیری: نامگذاری فقط label نیست؛ frame می‌سازد. وقتی می‌گوییم `Price-Late Selection`، در واقع یک اصل شناختی و محصولی را در نام ذخیره کرده‌ایم: بیمار نباید قبل از تشخیص داروی درست با قیمت anchor شود.

Medicine Selection Result

قاب داروسازی	قاب شناختی	قاب فنی
پاسخ انتخابی می‌گوید آیا دارو مشخص شده یا هنوز باید شکل، دوز یا هویت را روشن کنیم.	به جای اینکه بیمار با لیست خام مواجه شود، سیستم مرحله ابهام را تشخیص می‌دهد.	خروجی channel-neutral است؛ Telegram یا Bale فقط adapter نمایش دکمه‌اند.

Selection Session

قاب داروسازی	قاب شناختی	قاب فنی
وقتی بیمار چند مرحله انتخاب می‌کند، باید یادمان بماند از کجا شروع کرده بود.	بیمار نباید با callback بلند یا state آشکار درگیر شود.	یک store کوتاه‌عمر ۱۵ دقیقه‌ای sessionId را به query و choices وصل می‌کند.

TTAC Detail Enrichment

قاب داروسازی

عکس بسته، شکل ظاهری، ATC و کدها بعد از انتخاب دارو کمک‌کننده‌اند.

قاب شناختی

در مرحله اول انتخاب، این جزئیات بار شناختی زیاد می‌کنند.

قاب فنی

فعلاً contract placeholder داریم؛ fetch واقعی TTAC کار آینده است.

واژه	تعریف کوتاه	چرا مهم است
Consumer Unit Price	قیمت هر واحد مصرفی	بیمار قیمت قابل فهم می‌خواهد، نه قیمت خام بسته
Patient Form Label	برچسبی مثل پماد چشمی	نمایش ساده، مدل داخلی دقیق
Unknown Unit Choice	گزینه مطمئن نیستم	بیمار را به دانستن اصطلاح تخصصی مجبور نمی‌کند
Aggregate Selection History	یادگیری جمعی از انتخاب موفق	رایج بودن را از رفتار واقعی می‌آموزد

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۳ - زبان دامنه: چرا واژه‌ها معماری می‌سازند؟ را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۳ - زبان دامنه: چرا واژه‌ها معماری می‌سازند؟، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- واژه‌ها در این پروژه تصمیم معماری‌اند.
- هر اصطلاح باید بتواند در تجربه بیمار و کد معنا داشته باشد.
- واژه خوب جلوی feature اشتباه را می‌گیرد.

تمرین برای تثبیت

- برای `Price-Late Selection` یک مثال بیمارمحور بساز.
- توضیح بده چرا `Patient Form Label` با `dosageForm` یکی نیست.
- از روی واژه‌نامه بگو کدام اصطلاح‌ها به آینده TTAC مربوط‌اند.

فصل ۴ - قیمت واحد: قلب محصول بیمارمحور

بیمار اغلب نمی‌خواهد بداند قیمت فاکتور چطور حساب شده؛ می‌خواهد بداند «هر قرص چقدر می‌شود؟».

قیمت واحد، محور تجربه بیمار است. اگر دارو قرص است، پاسخ باید شبیه «قیمت هر قرص» باشد. اگر بطری است، «قیمت هر بطری». اگر بسته است و واحد مصرفی قابل استخراج نیست، باید صادق باشیم و واحد را روشن کنیم. این تصمیم ساده ظاهری، کل محصول را از نرم‌افزار داروخانه‌ای جدا می‌کند.

در اسناد پروژه تصمیم گرفته شد که فاز اولیه قیمت واحد را primary answer کند. این یعنی pack price می‌تواند وجود داشته باشد، ولی جواب اول برای بیمار نیست. اگر بیمار quantity بدهد، می‌توان total تخمینی هم داد، اما پایه همچنان unit price است.

چرا این مهم است؟ چون بیمار cognitive compression می‌خواهد. یک عدد با واحد روشن بهتر از سه عدد بدون context است. مخصوصاً وقتی بیمار سطح سواد سلامت پایین دارد، واحد روشن از خود عدد مهم‌تر می‌شود.

منطق قیمت

داروی انتخاب‌شده ← قیمت پایه Qanoon ← مالیات/عوارض/fee ← قیمت نهایی ← نمایش با واحد فارسی

قاعده طلایی قیمت

هیچ قیمت نهایی بیمارمحور نباید بدون نام واحد مصرفی نمایش داده شود.

در commit فعلی، متن نهایی Telegram هنوز کاملاً فارسی نشده و این یکی از next tasks است. یعنی منطق انتخاب چندمرحله‌ای آماده شده، اما عبارت نهایی مثل «قیمت هر قرص» باید در مرحله بعد تمیزتر شود.

حالت	پاسخ بد	پاسخ خوب
قرص	Unit price: 27000 per tablet	قیمت هر قرص: ۲۷,۰۰۰ ریال
بطری	Final price: 128000	قیمت هر بطری: ۱۲۸,۰۰۰ ریال
ابهام واحد	قیمت: ...	اول شکل دارو را انتخاب کنید

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۴ - قیمت واحد: قلب محصول بیمارمحور را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۴ - قیمت واحد: قلب محصول بیمارمحور، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- قیمت واحد زبان بیمار است.
- قیمت بسته و قیمت داخلی داروخانه‌ای نباید جواب اول باشند.
- واحد نامشخص یعنی هنوز برای قیمت نهایی زود است.

تمرین برای تثبیت

- برای یک داروی شربت، عبارت نهایی بیمارپسند بنویس.
- توضیح بده چرا قیمت بسته برای قرص ممکن است بیمار را گمراه کند.
- در کد فعلی کدام بخش باید برای فارسی‌سازی قیمت نهایی تغییر کند؟

فصل ۵ - `Qanoon Pharma` و `TTAC`: دو منبع، دو فلسفه

یکی برای قیمت و منطق داروخانه‌ای قوی‌تر است؛ دیگری برای هویت عمومی دارو و جزئیات بیمارپسند.

تصمیم ADR پروژه روشن است: در فاز بیمارمحور اولیه، قیمت نمایش‌داده‌شده از `Qanoon Pharma` می‌آید، چون منطق قیمت و داروخانه‌ای‌اش غنی‌تر است. `TTAC` برای هویت، برند، بسته‌بندی، وضعیت فعال، تصویر بسته، شکل ظاهری و اطلاعات بیشتر استفاده می‌شود.

این تفکیک خیلی مهم است. اگر `TTAC` قیمت بالاتری نشان دهد، به‌تنهایی باعث نمی‌شود قیمت اصلی را جایگزین کنیم. فقط اگر بتوانیم از بسته‌بندی `TTAC` با اطمینان unit price مشتق کنیم و اختلاف بیش از ۱۰,۰۰۰ ریال بود، warning ساده بیمارمحور نشان می‌دهیم.

از نگاه فلسفی، اینجا با دو ontology روبه‌روییم: ontology داروخانه و ontology بیمار. نرم‌افزار داروخانه قیمت، مالیات، سود، fee و stock می‌فهمد. بیمار جعبه، شکل، دوز، عکس و قیمت هر واحد را می‌فهمد. محصول باید بین این دو جهان bridge بسازد.

منبع	نقش اصلی	در جواب اولیه	در اطلاعات بیشتر
Qanoon Pharma	قیمت و منطق داروخانه	قیمت اصلی	source hint
TTAC	هویت و اطلاعات عمومی	کمک به selection	عکس، ATC، ترکیبات، کدها، مشابه‌ها
Payam Darousaz	مقایسه آینده	فعلاً ثانویه	احتمالاً warning یا cross-check

مرز مهم

TTAC را نباید به زور pricing engine کنیم، و Qanoon را نباید به زور patient catalog کنیم.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۵ - `Qanoon Pharma` و `TTAC`: دو منبع، دو فلسفه را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۵ - `Qanoon Pharma` و `TTAC`: دو منبع، دو فلسفه، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- `Qanoon Pharma` قیمت اصلی را می‌دهد.
- `TTAC` هویت و اطلاعات بیشتر را غنی می‌کند.
- اختلاف قیمت، warning است نه جایگزینی خودکار.

تمرین برای تثبیت

- اگر TTAC قیمت واحد بالاتری نشان داد، محصول چه باید بکند؟
- چرا عکس بسته باید بعد از انتخاب دارو بیاید؟
- یک مثال از داده TTAC بزن که برای selection مفید است اما برای قیمت اصلی کافی نیست.

فصل ۶ - انتخاب دارو قبل از قیمت: معماری جریان

اگر دارو مبهم است، سیستم باید سؤال خوب بپرسد؛ نه اینکه با اعتمادبه‌نفس قیمت اشتباه بدهد.

commit جدید module انتخاب دارو را اضافه کرد. این module تصمیم می‌گیرد خروجی `resolved` است یا هنوز `needs_strength_selection`، `needs_form_selection` یا `needs_identity_selection` لازم است. نکته مهم این است که این خروجی channel-neutral است. یعنی نه Telegram مالک منطق است، نه Bale؛ فقط adapter دکمه‌ای می‌سازند.

جریان معمول این است: بیمار query می‌فرستد. سیستم search می‌کند. اگر یک نتیجه واضح بود، resolved. اگر چند شکل دارویی بود، فرم را می‌پرسد. اگر شکل روشن شد اما چند دوز بود، دوز را می‌پرسد. اگر هنوز چند هویت مشابه باقی ماند، انتخاب identity-first می‌دهد. اگر بیمار گفت مطمئن نیستم، مستقیم به identity-first می‌رویم.

State machine انتخاب

query ← search ← needs_form_selection

needs_strength_selection ← needs_identity_selection ← resolved

نوع نتیجه	معنا	نمونه
resolved	دارو مشخص است	لوزارتان ۵۰ قرص
needs_form_selection	چند شکل باقی مانده	استامینوفن: قرص یا شیاف
needs_strength_selection	چند دوز باقی مانده	قرص ۳۲۵ یا ۵۰۰
needs_identity_selection	چند محصول مشابه باقی مانده	برند/بسته متفاوت
expired_selection	session قدیمی شده	دوباره نام دارو را بفرستید

چرا channel-neutral؟

چون قرار است همین منطق در وب، Bale ،Telegram و شاید WhatsApp کار کند. اگر منطق انتخاب را داخل دکمه Telegram بگذاریم، بعداً برای Bale دوباره‌کاری می‌شود.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۶ - انتخاب دارو قبل از قیمت: معماری جریان را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۶ - انتخاب دارو قبل از قیمت: معماری جریان، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- انتخاب دارو یک state machine ساده ولی عمیق است.
- adapterها فقط نمایش می‌دهند؛ منطق انتخاب در module مرکزی است.
- قیمت فقط بعد از resolved نمایش داده می‌شود.

تمرین برای تثبیت

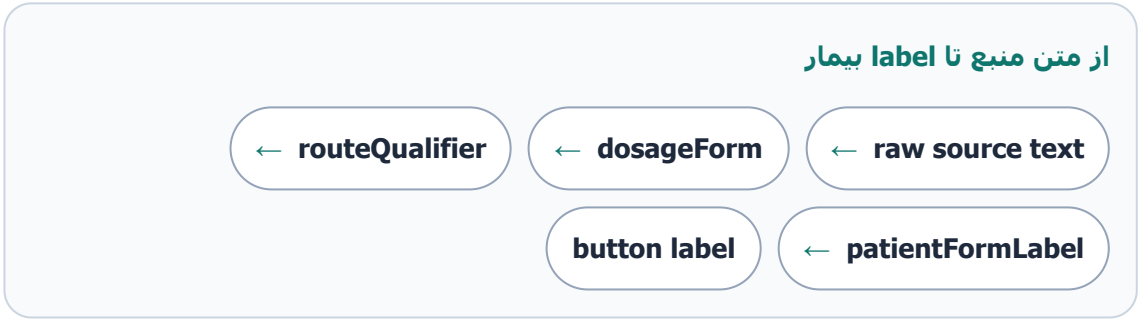
- برای query `استامینوفن` مسیر stateها را بنویس.
- توضیح بده چرا `مطمئن نیستم` باید به identity-first برود.
- اگر چند برند با دوز یکسان ماند، چه چیزی باید اول نمایش داده شود؟

فصل ۷ - شکل دارویی، route و label بیمار

`پماد چشمی` از نگاه بیمار یک عبارت است؛ از نگاه مدل داخلی دو جزء دارد.

یکی از تصمیم‌های خوب پروژه این بود که `dosageForm` و `routeQualifier` را جدا نگه داریم، اما برای بیمار label ترکیبی بسازیم. `پماد چشمی` برای بیمار یک گزینه طبیعی است. اما در مدل داخلی بهتر است `dosageForm=ointment` و `routeQualifier=ophthalmic` داشته باشیم. این جداسازی آینده را باز می‌کند.

چرا؟ چون `قطره چشمی` و `پماد چشمی` route مشترک دارند، اما form متفاوت. `قطره چشمی` و `قطره بینی` form مشترک دارند، اما route متفاوت. اگر همه چیز را فقط رشته خام کنیم، ranking و grouping زود خراب می‌شود.



عبارت بیمار/منبع	dosageForm	routeQualifier	نمایش بیمار
ophthalmic ointment	ointment	ophthalmic	پماد چشمی
nasal drop	drop	nasal	قطره بینی
sublingual tablet	tablet	sublingual	قرص زیرزبانی
oral syrup	syrup	oral	شربت

در این پروژه route اگر صریحاً در query بیمار باشد، ranking را قوی‌تر می‌کند اما form را کامل حدس نمی‌زند. اگر بیمار بگوید `ویتامین آ چشمی` و دو گزینه `پماد چشمی` و `قطره چشمی` وجود داشته باشد، باید هنوز انتخاب شکل بدهیم.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۷ - شکل دارویی، route و label بیمار را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۷ - شکل دارویی، route و label بیمار، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- نمایش بیمار می‌تواند ترکیبی باشد، اما مدل داخلی باید تفکیک‌شده بماند.
- route صریح انتخاب را محدود می‌کند، اما form را همیشه حدس نمی‌زند.
- این جداسازی باعث گسترش‌پذیری TTAC واقعی می‌شود.

تمرین برای تثبیت

- برای `قطره گوش` مدل داخلی و label بیمار را بنویس.
- چرا نباید هر عبارت ترکیبی را dosageForm مستقل بدانیم؟
- سه مثال از route qualifier فارسی بنویس.

فصل ۸ - شناخت بیمار: چرا ساده‌سازی یک feature است؟

ساده‌سازی اینجا کم‌عمق کردن نیست؛ طراحی شناختی برای کاهش خطا است.

در دارو، خطای انتخاب همیشه فقط UX بد نیست؛ می‌تواند به مصرف محصول اشتباه منجر شود. بنابراین اصل `Low-Literacy Selection` یکی از مهم‌ترین تصمیم‌های پروژه است. بیمار لازم نیست بداند dosage form یعنی چه. لازم نیست GTIN و IRC را در پاسخ اول ببیند. لازم نیست بین ده دکمه تخصصی انتخاب کند.

از نگاه علوم شناختی، سیستم باید working memory بیمار را محافظت کند. منوی اول حداکثر پنج گزینه اصلی دارد، همیشه `مطمئن نیستم` دارد، و اگر گزینه‌ها زیاد شد، به جای pagination عمیق، `جستجوی دقیق‌تر` پیشنهاد می‌کند. این یعنی product به جای نمایش قدرت داده، مسیر تصمیم را طراحی می‌کند.

همین جاست که دانش شناختی تو مزیت جدی می‌شود. می‌توانی هر feature را با این سؤال بسنجی: آیا این اطلاعات در این لحظه برای تصمیم بیمار لازم است؟ اگر نه، باید بعداً یا پشت «اطلاعات بیشتر» بیاید.

اصل	رفتار محصول	دلیل شناختی
کم‌گزینه	حداکثر پنج گزینه	کاهش بار حافظه کاری
مطمئن نیستم	مسیر بدون اجبار	کاهش شرم/بن‌بست شناختی
قیمت دیر	عدم نمایش قیمت در انتخاب اولیه	جلوگیری از anchor شدن روی ارزان‌ترین
اطلاعات بیشتر	TTAC detail بعد از resolved	جلوگیری از ازدحام ادراکی

فرمول UX دارویی

هرچه خطر انتخاب اشتباه بالاتر است، باید تمایزها کمتر، واضح‌تر و مرحله‌ای‌تر شوند.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۸ - شناخت بیمار: چرا ساده‌سازی یک feature است؟ را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۸ - شناخت بیمار: چرا ساده‌سازی یک feature است؟، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- ساده‌سازی در این پروژه محافظ ایمنی است.
- `مطمئن نیستم` یک مسیر شناختی مهم است.
- اطلاعات غنی همیشه اطلاعات مفید در لحظه نیست.

تمرین برای تثبیت

- برای یک بیمار با سواد سلامت پایین، منوی بد و منوی خوب طراحی کن.
- توضیح بده چرا قیمت زودهنگام می‌تواند انتخاب را خراب کند.
- یک feature را پیدا کن که باید به جای پاسخ اول، در اطلاعات بیشتر باشد.

فصل ۹ - یادگیری جمعی: محبوبیت، اما نه کور

سیستم باید از انتخاب‌های واقعی یاد بگیرد، اما حرف صریح بیمار را شکست ندهد.

در بحث `ویتامین آ`، حدس اولیه من اشتباه بود: معمولاً ممکن است پماد چشمی موردنظر باشد، نه قرص. این خطا دقیقاً نشان داد چرا نباید popularity را hard-code کنیم. محصول باید از انتخاب‌های موفق قبلی یاد بگیرد، نه از حافظه یا شهود ما.

اما یادگیری هم خطر دارد. اگر بیمار صریح بنویسد `قرص ویتامین آ`، حتی اگر تاریخچه نشان دهد بیشتر کاربران پماد چشمی خواسته‌اند، قرص باید جلوتر بیاید. بنابراین ranking priority این است: متن صریح بیمار، active status و هویت روشن، learned selection history، سپس کامل بودن دوز و بسته.

یادگیری فاز اول aggregate و غیرشخصی است. یعنی پروفایل بیمار نمی‌سازیم. فقط می‌فهمیم برای queryهای مبهم، قبلاً کدام انتخاب‌ها به قیمت واحد نهایی رسیده‌اند. selection فقط وقتی موفق است که resolved شود و قیمت نهایی نشان داده شود.

ثبت یادگیری

query مبهم ← انتخاب بیمار ← resolved ← نمایش قیمت واحد ←

ثبت در Aggregate Selection History

سیگنال ranking	قدرت	نمونه
متن صریح بیمار	بسیار بالا	قرص، چشمی، ۵۰۰
active status	بالا	نتیجه فعال TTAC
یادگیری جمعی	متوسط	ویتامین آ -> پماد چشمی
کامل بودن داده	کمکی	دوز و بسته واضح

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۹ - یادگیری جمعی: محبوبیت، اما نه کور را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۹ - یادگیری جمعی: محبوبیت، اما نه کور، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- یادگیری باید از رفتار واقعی بیاید.
- محبوبیت هرگز متن صریح بیمار را شکست نمی‌دهد.
- در فاز اول یادگیری شخصی نداریم.

تمرین برای تثبیت

- برای query `ویتامین آ` دو سناریوی ranking بنویس: با متن صریح و بدون متن صریح.
- چرا abandoned click نباید وارد یادگیری شود؟
- چه زمانی یک انتخاب successful است؟

فصل ۱۰ - Session و callback: حافظه کوتاه بات

دکمه بات باید کوتاه باشد؛ مغز state باید سمت سیستم بماند.

در bot flow، اگر همه اطلاعات choices، filters، query و state را داخل callback بگذاریم، payload طولانی و شکننده می‌شود. تصمیم پروژه این است که callback کوتاه باشد: `ms:<sessionId>:<choiceId>`. سپس session store سمت سیستم می‌داند این session مربوط به چه query و چه choices بوده است.

در commit فعلی، `selection-session-store.ts` یک store in-memory دارد. برای development خوب است، اما production نیست. وقتی وارد `bale-med` شویم، احتمالاً باید session را در SQLite یا store durable مناسب نگه داریم، چون بات واقعی restart می‌شود و کاربر ممکن است بعد از چند دقیقه دکمه بزند.

مدت session پیش‌فرض ۱۵ دقیقه است. اگر session منقضی شد، بات نباید silent fail کند. باید ساده بگوید: «این جستجو قدیمی شده. لطفاً نام دارو را دوباره بفرستید.» این یک خط ساده است، اما از نظر تجربه بیمار خیلی مهم است.




```
callback_data = "ms:<sessionId>:<choiceId>"
مثال:
ms:sel-1:form:tablet
ms:sel-2:strength:۵۰۰ میلی‌گرم
```

آینده در bale-med

چون `bale-med` SQLite دارد، session و selection history در آینده می‌توانند کنار incoming_messages یا در جدول‌های جدا ذخیره شوند.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۰ - Session و callback: حافظه کوتاه بات را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۰ - Session و callback: حافظه کوتاه بات، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- callback کوتاه است، state در session می‌ماند.
- session فعلی development-only و in-memory است.
- برای production باید durable شود.

تمرین برای تثبیت

- اگر app restart شود، session in-memory چه مشکلی پیدا می‌کند؟
- یک schema ساده برای جدول selection_sessions پیشنهاد کن.
- چرا خطای session منقضی نباید technical باشد؟

فصل ۱۱ - `TTAC Detail Enrichment`: اطلاعات بیشتر، نه شلوغی بیشتر

TTAC گنج اطلاعاتی است، ولی زمان نمایش هر گنج مهم است.

صفحه detail در `TTAC` اطلاعات جذابی دارد: عکس بسته، عکس شکل ظاهری، ATC، ترکیبات، قیمت مصرف‌کننده، IRC، GTIN و محصولات مشابه. وسوسه طبیعی این است که همه را وارد selection کنیم. اما تصمیم درست فعلی: جزئیات غنی بعد از resolved می‌آیند.

چرا؟ چون بیمار در مرحله انتخاب اولیه باید یک تمایز ساده انجام دهد: قرص یا شیاف؟ ۵۰۰ یا ۳۲۵؟ پماد چشمی یا قطره چشمی؟ اگر همان ابتدا ATC tree و عکس و کدها را نشان بدهیم، cognitive overload می‌سازیم. ولی بعد از انتخاب دارو، عکس بسته و شکل ظاهری می‌تواند اعتماد و تشخیص را بالا ببرد.

در خروجی فعلی `moreInfo.ttacDetail = null` گذاشته شده. این یعنی contract ذهنی آماده است، اما fetch واقعی هنوز انجام نشده. گام آینده می‌تواند یک adapter مستقل باشد که با `ttacCode` یا detail URL اطلاعات را fetch کند و به result resolved اضافه کند.

TTAC detail	زمان نمایش	نقش
عکس بسته	بعد از resolved	تأیید بصری
شکل ظاهری	بعد از resolved	کمک به تشخیص دارو
ATC	اطلاعات بیشتر	فنی/طبقه‌بندی
ترکیبات	اطلاعات بیشتر	آگاهی، نه تصمیم درمانی
مشابه‌ها	بعد از resolved	مقایسه هویتی، نه جایگزینی خودکار

اصل TTAC

TTAC برای غنی‌سازی هویت عالی است؛ برای قیمت اصلی یا توصیه درمانی نباید بی‌محابا استفاده شود.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۱ - `TTAC Detail Enrichment`: اطلاعات بیشتر، نه شلوغی بیشتر را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۱ - `TTAC Detail Enrichment`: اطلاعات بیشتر، نه شلوغی بیشتر، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- TTAC detail بعد از انتخاب دارو می‌آید.
- عکس‌ها فعلاً URL/metadata هستند، نه دانلود دائمی.
- مشابه‌ها ranking اولیه را تغییر نمی‌دهند.

تمرین برای تثبیت

- یک UI کوچک برای «اطلاعات بیشتر از تیتک» تصور کن.
- چرا محصولات مشابه نباید داروی انتخاب‌شده را جایگزین کنند؟
- چه اطلاعات TTAC برای بیمار عمومی کم‌فایده یا گیج‌کننده است؟

فصل ۱۲ - Git برای این پروژه: از حافظه تا تاریخچه

Git فقط ابزار ذخیره نیست؛ دفترچه آزمایش‌های فکری پروژه است.

برای Git ،vibe coding نقش مهم‌تری از backup دارد. هر commit باید یک تغییر مفهومی را ثبت کند. commit اول این پروژه payload دکمه‌ای بات را اضافه کرد. commit دوم flow انتخاب دارو را اضافه کرد. این یعنی تاریخچه Git مثل فصل‌های رشد محصول خوانده می‌شود.

قاعده عملی: قبل از تغییر، وضعیت را ببین. بعد تغییر کوچک بزن. تست بگیر. diff را بخوان. commit کن. اگر قرار است به GitHub و `bale-med` وصل شوی، branch بساز، PR باز کن، review/check بگیر، merge کن، سپس main را sync کن.

```
git status --short
git diff --stat
git diff
git add <files>
git commit -m "Add patient medicine selection flow"
git log --oneline -5
```

فرمان	معنا برای ذهن تو	زمان استفاده
git status --short	الان چه چیزهایی تغییر کرده؟	قبل و بعد از هر کار
git diff	دقیقاً چه چیزی عوض شد؟	قبل از commit
git add	این تغییرها را برای ثبت انتخاب کن	وقتی scope روشن است
git commit	یک تصمیم/گام را در تاریخچه ثبت کن	بعد از تست سبز
git log --oneline	داستان پروژه را مرور کن	برای orientation

در `README` ، `bale-med` پیشنهاد می‌کند milestoneها با branch و PR کوچک جلو بروند. این برای پروژه دارویی خوب است، چون safety و auditability مهم‌اند. تغییرهای بزرگ، مخصوصاً وقتی به بیمار و داروساز مربوط‌اند، باید traceable باشند.

Git برای داروساز پژوهشگر

هر commit مثل یک observation در دفتر آزمایش است: فرضیه، تغییر، نتیجه تست. اگر این سه را از commit بفهمی، تاریخچه پروژه آموزشی می‌شود.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۲ - Git برای این پروژه: از حافظه تا تاریخچه را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۲ - Git برای این پروژه: از حافظه تا تاریخچه، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- Git حافظه اجرایی پروژه است.
- commit خوب یک تغییر مفهومی دارد.
- برای `bale-med` بهتر است branch/PR کوچک و milestone-aligned باشد.

تمرین برای تثبیت

- آخرین دو commit پروژه را مثل دو جمله روایی توضیح بده.
- قبل از commit چه سه چیزی را باید چک کنی؟
- یک commit message خوب برای فارسی‌سازی قیمت نهایی پیشنهاد کن.

فصل ۱۳ - ابزارها: GitHub، Chrome، Codex، PDF

ابزار خوب وقتی خوب استفاده می‌شود که بدانی کدام سؤال را از کدام ابزار بپرسی.

در این پروژه چند ابزار نقش داشتند. `Chrome` برای دیدن TTAC و فهم صفحه واقعی. `GitHub` برای خواندن `Git`. `bale-med` برای ثبت تغییر. تست‌ها برای جلوگیری از شکستن رفتار. مهارت‌های `grill-with-docs` برای تبدیل ایده‌های مبهم به تصمیم‌های ثبت‌شده. مهارت `implement` برای تبدیل تصمیم به کد.

منطق استفاده از ابزارها این است: اگر سؤال درباره واقعیت بیرونی است، نگاه کن. اگر درباره repo است، بخوان و جستجو کن. اگر درباره tradeoff محصول است، grill کن. اگر تصمیم گرفته شد، implement کن. اگر خروجی آموزشی یا artifact می‌خواهی، ابزار مناسب ساخت artifact را انتخاب کن.

ابزار/مهارت	کار درست	خطای رایج
Chrome	مشاهده صفحه TTAC واقعی	حدس زدن ساختار منبع بدون دیدن
GitHub	مطالعه bale-med و workflow	فرض اینکه پروژه مقصد شبیه پروژه فعلی است
grill-with-docs	ثبت تصمیم‌های دامنه	کد زدن قبل از روشن شدن واژه‌ها
implement	کد، تست، commit، review	پیشنهاد دادن بدون اجرا
PDF	ساخت artifact خواندنی و قابل مرور	تحویل متن خام بدون بررسی visual

انتخاب ابزار

سؤال چیست؟ ← **واقعیت repo؟** ← **واقعیت وب؟** ←

tradeoff محصول؟ ← **کد؟** ← **artifact؟**

برای ایده‌سازی آینده، ابزارها را مثل extension شناختی ببین. تو مدل انتزاعی می‌سازی؛ ابزارها کمک می‌کنند مدل را با واقعیت برخورد بدهی. Chrome واقعیت TTAC را نشان می‌دهد. GitHub واقعیت `bale-med` را نشان می‌دهد. تست واقعیت رفتار کد را نشان می‌دهد.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۳ - ابزارها: Codex، PDF، GitHub، Chrome را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۳ - ابزارها: PDF، GitHub، Chrome، Codex، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- ابزارها برای کاهش توهم و افزایش locality استفاده می‌شوند.
- هر ابزار سؤال مناسب خودش را دارد.
- vibe coding خوب یعنی vibe + verification.

تمرین برای تثبیت

- برای فهم یک feature جدید TTAC، کدام ابزارها را به چه ترتیب استفاده می‌کنی؟
- چه زمانی grill بهتر از implement است؟
- چه زمانی تست از توضیح شفاهی مهم‌تر است؟

فصل ۱۴ - تست‌ها: قرارداد رفتاری، نه جزئیات پیاده‌سازی

تست خوب می‌گوید کاربر چه رفتاری می‌بیند، نه اینکه تابع داخلی چه نامی دارد.

در commit جدید، تست‌های `bot-lookup-service.test.ts` مسیرهای مهم را پوشش می‌دهند: تک‌نتیجه resolved می‌شود؛ استامینوفن چندشکلی form selection می‌دهد؛ انتخاب قرص به strength selection می‌رود؛ انتخاب قدرت به قیمت واحد resolved می‌شود؛ query بدون نتیجه not_found می‌دهد.

این سبک تست برای پروژه خوب است چون از public-ish interface یعنی `botLookupService` استفاده می‌کند. اگر فردا داخل `medicine-selection-service.ts` را refactor کنیم، تا وقتی رفتار بیرونی همان باشد، تست‌ها باید پاس بمانند. این همان ایده «interface is the test surface» است.

```
npm test

رفتارهای مهم:
- lookup single match -> resolved
- ambiguous medicine -> needs_form_selection
- form choice -> needs_strength_selection
- strength choice -> resolved
- missing query -> not_found
```

وقتی این پروژه وارد `bale-med` شود، تست‌ها باید در سطح route یا handler هم اضافه شوند. مثلاً callback parsing در `session expired/`، `bot/select`، و safety boundary با high-risk message. تست قیمت نباید باعث شود مسیر review داروساز دور زده شود.

تست	چرا مهم است
form -> strength -> resolved	اثبات مسیر چندمرحله‌ای
expired session	جلوگیری از سکوت گیج‌کننده
price hidden in early selection	حفظ Price-Late Selection
high-risk message in bale-med	حفظ safety boundary
TTAC detail placeholder	آمادگی آینده بدون fetch واقعی

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۴ - تست‌ها: قرارداد رفتاری، نه جزئیات پیاده‌سازی را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۴ - تست‌ها: قرارداد رفتاری، نه جزئیات پیاده‌سازی، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- تست‌ها باید رفتار بیمار/بات را حفظ کنند.
- refactor داخلی نباید تست رفتاری را بشکند.
- ادغام با `bale-med` نیازمند تست safety boundary است.

تمرین برای تثبیت

- یک تست برای callback expired بنویس، حتی به زبان فارسی.
- چرا تست private function شکننده است؟
- برای ادغام قیمت‌یاب با `bale-med` سه تست ضروری پیشنهاد کن.

فصل ۱۵ - مسیر ادغام با `bale-med`

قیمت‌یاب باید به شکل یک capability وارد بات شود، نه اینکه بات را از نو تعریف کند.

ادغام ایده‌آل این است که `bale-med` همچنان مالک reply safety، database، security، webhook و pharmacist review باشد. قیمت‌یاب به‌عنوان یک capability زیرمجموعه وارد شود: وقتی intent پیام قیمت/داروی مشخص است، route_update می‌تواند پیام را به selection/pricing engine بسپارد و پاسخ دکمه‌ای بسازد.

در `bale-med` مسیر فعلی پیام این است: webhook -> `route_update` -> `IncomingMessage.from_update` -> `save_incoming_message` -> safe reply یا escalation. قیمت‌یاب باید جایی اضافه شود که پس از ذخیره و قبل از reply نهایی بتواند intent را بررسی کند، اما high-risk rules را دور نزند.

ادغام پیشنهادی در bale-med

webhook ← parse IncomingMessage ← save message

safety/high-risk check ← price intent? ← medicine selection

Bale reply/buttons ← review if needed

تصمیم ادغام	پیشنهاد
محل logic	module جدا، نه داخل handlers.py به شکل بزرگ
storage	SQLite برای session/history در آینده
safety	high-risk قبل یا همزمان با intent قیمت بررسی شود
reply	فارسی، کم‌کلمه، بدون advice درمانی
TTAC	adapter جدا و post-resolution

اصل سازگاری

هر feature جدید قیمت باید با جمله `README` `bale-med` سازگار باشد: این bot تشخیص، نسخه، دوز، تغییر دارو یا جایگزینی داروساز نیست.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۵ - مسیر ادغام با `bale-med` را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۵ - مسیر ادغام با `bale-med`، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- `bale-med` میزبان workflow است، pricing engine capability مهمان است.
- safety boundary باید بالاتر از convenience باشد.
- ادغام خوب با module جدا و تست route/handler انجام می‌شود.

تمرین برای تثبیت

- یک pseudo-flow برای message `قیمت استامینوفن` در `bale-med` بنویس.
- یک پیام high-risk بنویس و توضیح بده چرا نباید وارد price flow شود.
- اسم یک module Python برای اتصال selection به Bale پیشنهاد کن.

فصل ۱۶ - ایده‌سازی آینده: از فهم تا خلق feature

وقتی منطق را بفهمی، feature جدید دیگر حدس نیست؛ امتداد مدل است.

برای تولید ایده جدید، از سه محور استفاده کن: محور بیمار، محور داروساز، محور منبع داده. محور بیمار می‌پرسد: بیمار کجا گیج می‌شود؟ محور داروساز می‌پرسد: کجا باید review انسانی باشد؟ محور منبع داده می‌پرسد: TTAC/Qanoon چه چیزی می‌دهند و چه چیزی نمی‌دهند؟

مثلاً feature «مشاهده عکس دارو» از محور بیمار می‌آید: بیمار ممکن است جعبه یا قرص را ببیند و اعتماد بگیرد. اما چون عکس‌ها از TTAC می‌آیند و رسانه policy دارند، فعلاً URL/metadata نگه می‌داریم. feature «warning اختلاف قیمت» از محور منبع داده می‌آید. feature «ارجاع به داروساز» از محور safety داروساز می‌آید.

ایده	محور	ریسک	گام امن
عکس بسته	بیمار/TTAC	حجم و حق نشر	لینک بعد از resolved
قیمت مشابه‌ها	منبع داده	جایگزینی اشتباه	فقط اطلاعات بیشتر
ارسال به داروساز	داروساز	workflow ناقص	review metadata و notification
یادگیری محبوبیت	شناختی/داده	bias	aggregate و non-personal

قاعده ایده‌سازی: هر ایده را اول به واژه‌نامه دامنه وصل کن. اگر واژه ندارد، یا ایده خام است یا مدل دامنه نیاز به واژه جدید دارد. سپس برایش مسیر تست بنویس. بعد بپرس در `bale-med` کجا می‌نشیند و آیا safety boundary را لمس می‌کند یا نه.

فرمول تولید feature

ابهام بیمار + داده قابل اعتماد + مرز ایمنی + تست رفتاری = feature سالم.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۶ - ایده‌سازی آینده: از فهم تا خلق feature را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۶ - ایده‌سازی آینده: از فهم تا خلق feature، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- ایده جدید باید امتداد مدل باشد.
- هر feature باید محور بیمار، داروساز و منبع داده را چک کند.
- واژه‌نامه و تست، ابزار ایده‌سازی‌اند.

تمرین برای تثبیت

- سه ایده برای TTAC detail enrichment بنویس.
- برای هر ایده بگو کدام safety risk دارد.
- یکی از ایده‌ها را به یک تست رفتاری تبدیل کن.

فصل ۱۷ - چک‌لیست تسلط: اگر خواستی به Codex درس بدهی

هدف کتاب این است که دفعه بعد تو راهبر مدل باشی، نه فقط درخواست‌دهنده.

وقتی دوباره سراغ من آمدی، می‌توانی با این جمله شروع کنی: «ما یک موتور انتخاب و قیمت واحد دارو داریم که در نهایت capability داخل `bale-med` می‌شود. قیمت از Qanoon می‌آید، هویت از TTAC غنی می‌شود، selection قبل از price است، و `bale-med` `safety boundary` نباید دور زده شود.» اگر این جمله را راحت بگویی، ستون فقرات پروژه را گرفته‌ای.

برای vibe coding قوی، لازم نیست همه جزئیات syntax را حفظ کنی. باید بدانی سؤال درست چیست، فایل مسئول کجاست، تست رفتاری چیست، و خطر محصولی کجاست. این‌ها همان نقاطی‌اند که یک داروساز با زمینه شناختی می‌تواند از یک developer صرف جلوتر باشد.

اگر می‌خواهی...	به من بگو...	من باید...
قیمت نهایی فارسی شود	قیمت هر قرص/بطری را فارسی کن	pricing text را refactor و تست کنم
TTAC detail اضافه شود	بعد از resolved عکس و کدها را contract کن	adapter جدا طراحی کنم
به bale-med وصل شود	capability قیمت را داخل safety flow بگذار	handler/route امن بسازم
یادگیری durable شود	history/session را از memory به SQLite ببر	migration و تست اضافه کنم

جمله راهبر

هر وقت feature جدید خواستی، اول بگو: «این feature در کدام مرحله از مسیر بیمار است: قبل از انتخاب، هنگام انتخاب، بعد از قیمت، یا review داروساز؟»

- آیا قیمت قبل از هویت دارو نمایش داده می‌شود؟ اگر بله، خطرناک است.
- آیا اصطلاح تخصصی بدون نیاز به بیمار داده شده؟ اگر بله، ساده‌تر کن.
- آیا TTAC detail وارد منوی اولیه شده؟ اگر بله، احتمال overload دارد.
- آیا high-risk message وارد price flow شده؟ اگر بله، safety boundary شکسته.

- آیا تغییر بدون تست رفتاری است؟ اگر بله، هنوز آماده نیست.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۷ - چک‌لیست تسلط: اگر خواستی به Codex درس بدهی را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۷ - چک‌لیست تسلط: اگر خواستی به Codex درس بدهی، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- تو باید بتوانی پروژه را با یک جمله راهبر توضیح بدهی.
- فهم architecture از حفظ کد مهم‌تر است.
- سؤال درست، ابزار درست را انتخاب می‌کند.

تمرین برای تثبیت

- جمله راهبر خودت را برای پروژه بنویس.
- یک feature آینده انتخاب کن و مرحله بیمار آن را مشخص کن.
- برای feature انتخابی، یک تست رفتاری و یک خطر safety بنویس.

فصل ۱۸ - ضمیمه: واژه‌نامه آینده و پک‌لیست‌ها

این فصل برای مرور سریع بعد از مطالعه طولانی است.

عبارت	ترجمه/حس فارسی	استفاده آینده
Consumer Unit Price	قیمت هر واحد مصرفی	متن نهایی قیمت
Selection Session	حافظه کوتاه انتخاب	callback چندمرحله‌ای
Identity-First Selection	اول هویت دارو	وقتی بیمار مطمئن نیست
Price-Late Selection	قیمت دیرتر	جلوگیری از انتخاب اشتباه
TTAC Detail Enrichment	اطلاعات تکمیلی تیتک	بعد از resolved
Aggregate Selection History	یادگیری جمعی	ranking queryهای مبهم
Route Qualifier	قید مسیر مصرف	چشمی، گوشی، بینی
Patient Form Label	برچسب بیمارخوان شکل	پماد چشمی، قطره بینی

پک‌لیست مطالعه دوباره

- اول فصل‌های ۱ تا ۳ را برای مدل ذهنی بخوان.
- بعد فصل‌های ۶ تا ۱۰ را برای فهم flow و کد بخوان.
- اگر هدف ادغام با `bale-med` است، فصل ۲ و ۱۵ را دوباره بخوان.
- قبل از درخواست feature جدید، فصل ۱۶ و ۱۷ را مرور کن.
- قبل از commit، فصل Git و تست را نگاه کن.

پک‌لیست توسعه بعدی

- آیا feature داخل `pricing-app` است یا `bale-med`؟
- آیا safety boundary لمس می‌شود؟
- آیا نیاز به glossary term جدید دارد؟
- آیا test behavior مشخص است؟
- آیا خروجی بیمار فارسی، ساده و کم‌بار است؟
- آیا Git branch/commit کوچک و قابل review است؟

اگر فقط یک چیز از این کتاب نگه داری، این باشد: پروژه دارویی خوب فقط داده را نشان نمی‌دهد؛ مسیر تصمیم را طراحی می‌کند. مسیر تصمیم یعنی چه چیزی را الان نشان بدهیم، چه چیزی را بعداً، چه چیزی را هرگز، و چه چیزی را فقط داروساز باید ببیند.

لایه عمیق‌تر: چگونه این فصل را مثل یک مدل ذهنی نگه داری؟

برای تندخوانی سه‌ساعته، هدف این نیست که جمله‌های این فصل ۱۸ - ضمیمه: واژه‌نامه آینده و پک‌لیست‌ها را حفظ کنی. هدف این است که اسکلت استدلال را در ذهن بسازی. اول بپرس این فصل چه تمایزی می‌سازد؛ بعد بپرس این تمایز در رفتار بیمار چه اثری دارد؛ بعد بپرس در کد کدام module یا interface باید این تمایز را نگه دارد. این سه پرسش، متن را از اطلاعات پراکنده به ابزار فکر تبدیل می‌کند.

از نگاه شناختی، یادگیری عمیق وقتی رخ می‌دهد که مفهوم را در چند سطح بازنمایی کنی. یک بار آن را با زبان بیمار بگو، یک بار با زبان داروساز، یک بار با زبان داده، و یک بار با زبان کد. اگر یک مفهوم فقط در یکی از این زبان‌ها معنا دارد، هنوز برای این پروژه پخته نشده است. مفهوم خوب باید بتواند از نسخه و جعبه دارو تا endpoint و تست رفتاری حرکت کند.

وقتی بعداً خواستی feature تازه بسازی، این فصل را مثل دستگاه تولید سؤال استفاده کن. سؤال اول: بیمار در این نقطه چه چیزی را نمی‌داند؟ سؤال دوم: داروساز در این نقطه چه چیزی را نباید به ماشین واگذار کند؟ سؤال سوم: داده موجود چه چیزی را با اطمینان می‌گوید و چه چیزی را فقط القا می‌کند؟ سؤال چهارم: اگر این feature اشتباه کار کند، آیا فقط UX بد می‌شود یا خطر دارویی هم داریم؟

در vibe coding، خطر اصلی این نیست که syntax را فراموش کنی؛ خطر این است که مدل ذهنی را به کد ترجمه نکنی. قبل از هر prompt به Codex، یک جمله راهبر بنویس: «این تغییر باید کدام ابهام بیمار را کم کند، بدون اینکه safety boundary را دور بزند؟» این جمله باعث می‌شود خروجی مدل به سمت feature سالم برود، نه فقط کد قابل اجرا.

برای مرور آخر شب، همین فصل را با تکنیک teach-back بخوان: فرض کن من هیچ چیز نمی‌دانم و تو باید در پنج دقیقه توضیح بدهی چرا این مفهوم در پروژه مهم است. اگر وسط توضیح به کلمه انگلیسی رسیدی، آن را تعریف کن. اگر به فایل رسیدی، بگو چرا آن فایل محل طبیعی آن مفهوم است. اگر به آینده رسیدی، بگو کدام تست باید جلوی خراب شدنش را بگیرد.

لایه	سؤال راهبر	نشانه فهم
بیمار	بیمار چه می‌بیند یا نمی‌فهمد؟	می‌توانی یک مثال فارسی ساده بزنی
داروساز	کدام بخش نیاز به قضاوت انسانی دارد؟	می‌توانی مرز advice را توضیح بدهی
شناختی	بار تصمیم کجاست؟	می‌توانی بگویی چه چیزی باید پنهان یا مرحله‌ای شود
فنی	کدام module مالک رفتار است؟	می‌توانی interface و تست رفتاری را نام ببری
آینده	اگر توسعه دهیم چه چیزی می‌شکند؟	می‌توانی migration یا adapter بعدی را تصور کنی

prompt آماده برای جلسه بعد

«در زمینه این فصل ۱۸ - ضمیمه: واژه‌نامه آینده و پک‌لیست‌ها، اول مدل دامنه را چک کن، بعد بگو این تغییر قبل از انتخاب دارو است یا بعد از resolved، بعد تست رفتاری پیشنهاد بده، بعد implement کن.»

خلاصه انتهای فصل

- واژه‌نامه ابزار فکر است.
- پک‌لیست‌ها کمک می‌کنند پروژه بعد از وقفه دوباره سریع وارد ذهن شود.
- هدف نهایی: تولید ایده سالم و قابل پیاده‌سازی.

تمرین برای تثبیت

- سه اصطلاح را انتخاب کن و با مثال دارویی توضیح بده.
- یک feature آینده را با پک‌لیست توسعه بعدی ارزیابی کن.
- برای جلسه بعد، یک prompt راهبر برای Codex آماده کن.